# A polynomial time algorithm for solving the bi-objective spanning star forest problem on cactus graphs

Thanh Loan Nguyen[1] and Viet Hung Nguyen[1*]

[1]LIMOS (UMR 6158), INP Clermont Auvergne, Université Clermont Auvergne, Mines Saint-Étienne, CNRS, 1 Rue de la Chebarde, Aubière, 63178, France.

*Corresponding author(s). E-mail(s): viet_hung.nguyen@uca.fr;
Contributing authors: thanh_loan.nguyen@doctorant.uca.fr;

**Abstract**

Given an undirected graph $\boldsymbol{G = (V, E)}$ where the edges are weighted positively, a star in $\boldsymbol{G}$ is a connected subgraph of $\boldsymbol{G}$ of diameter at most 2. When a star contains at least two edges of $\boldsymbol{G}$, the center of this star is its unique vertex of degree at least 2, otherwise, the center can be any of its vertices. A spanning star forest in $\boldsymbol{G}$ is a collection of vertex disjoint stars covering all the vertices of $\boldsymbol{G}$. This paper investigates the Bi-objective Spanning Star Forest Problem (BSSFP), where the goal is to minimize both the total edge weight and the number of centers in a spanning star forest of a graph. This problem is NP-hard on general graphs and was first introduced in [10], where a polynomial time algorithm was proposed for solving the BSSFP on trees. In this paper, we consider the BSSFP on cactus graphs, a graph class generalizing trees where an edge belongs to at most a cycle. We propose an exact two-phase dynamic programming algorithm of time complexity $\boldsymbol{O(|E||V|^2)}$ based on a tree-cycle decomposition of a cactus graph, for enumerating the efficient solutions of the BSSFP. Subproblems on tree components are solved using dynamic programming, while cycle components are handled by combining the corresponding dynamic programming states using a min-plus product. The latter is a new contribution making our result a strict extension of the previous result in [10].

**Keywords:** Bi-objective Combinatorial Optimization, Spanning Star Forest, Cactus graph, Dynamic programming, Min-plus product

# 1 Introduction

The Spanning Star Forest (SSF) problem has recently attracted attention in combinatorial optimization that naturally relates to both facility location and network design [3, 4]. Let $G = (V, E)$ be an undirected graph. A subgraph $S$ of $G$ is a *star* if either $S$ consists of an isolated node, or there exists a node $v \in V(S)$ such that every edge of $S$ is incident to $v$. Node $v$ is then called the *center* of $S$. A subgraph $F$ of $G$ is a *star forest* if each connected component of $F$ is a star. A *SSF* of $G$ is a star forest $F$ such that $V(F) = V$. Let $w \in \mathbb{R}_+^{|E|}$ be the edge-weight vector of $G$, the weight of a SSF is defined as the sum of the weights of its edges. Over the past decade, the maximization version of the SSF problem has been extensively studied. Nguyen et al. [9] prove that the maximum weighted SSP on general graphs is an NP-hard problem, and they design a linear-time algorithm for this problem on trees. This was subsequently extended by Nguyen [11] to cactus graphs, a class of graphs where any two cycles share at most one node. In contrast, the minimization version of the SSF problem presents a fundamentally different structural challenge. Unlike the maximization version, minimizing the total weight is trivial unless the number of star centers, denoted by $k$, is treated as a constraint. This variant is motivated by industrial applications. Agra et al. [1] investigated this problem in the context of automobile configuration systems and proved that minimizing the total weight with a constraint of at most $k$ centers is NP-hard on general graphs. In a related work, Briant et al. [5] studied the Optimal Diversity Management Problem, where exactly $k$ centers are selected, and showed its NP-hardness via a reduction to the $p$-median problem.

An aspect shared by the above studies is that the number of centers $k$ is assumed to be fixed in advance. In this work, rather than focusing on a predetermined value of $k$, we consider a context in which the trade-off between the number of centers and the total weight is explored explicitly. This leads to the *Bi-objective Spanning Star Forest Problem (BSSFP)* [10], which minimizes both the number of centers and the total weight of SSFs. Given an edge-weighted graph $G = (V, E, w)$ with the assumption that the weight vector $w$ consists of positive integers, the BSSFP can be formulated as

$$\min_{x \in \mathcal{X}} (f_1(x), f_2(x)),$$

where $f_1(x)$ denotes the total weight, $f_2(x)$ denotes the number of centers, and $\mathcal{X}$ is the set of all feasible decision vectors associated with SSFs of $G$. Let $(f_1, f_2) = (f_1(x), f_2(x))$ denote the objective pair induced by a feasible vector $x$, and let $\mathcal{F}$ be the set of all such pairs. Throughout this paper, feasible solutions of the BSSFP are represented by their objective pairs rather than by explicit decision vectors, and two solutions inducing the same pair $(f_1, f_2)$ are considered equivalent. As a special case of a bi-objective optimization problem, a solution $(f_1, f_2) \in \mathcal{F}$ is called *efficient* (i.e., *Pareto optimal*) if there exists no other solution $(f_1', f_2') \in \mathcal{F}$ such that $f_1' \leq f_1$ and $f_2' \leq f_2$, with at least one inequality being strict. Geometrically, efficient solutions are classified into *supported efficient solutions*, which are solutions that can be obtained as optimal solutions of weighted sum scalarizations and lie on the boundary of the

convex hull of $\mathcal{F}$, and *non-supported efficient solutions*, which correspond to efficient solutions lying in the interior of this convex hull.

Observe that the objectives of the BSSFP are conflicting. Indeed, minimizing the total weight $f_1$ yields a trivial solution in which all nodes are isolated, with total weight equal to zero, and the number of centers achieves the maximum value $f_2 = |V|$. By contrast, minimizing the number of centers $f_2$ tends to increase the total weight and may lead to a maximum-weight SSF. Since the maximum version is NP-hard on general graphs [9], it follows that the BSSFP is also NP-hard. Observe that approaches based on fixing the number of centers $k$, as considered in previous works, do not guarantee that the obtained solutions are Pareto optimal. In contrast, the BSSFP is designed for situations in which the decision maker wishes to retain flexibility in the choice of $k$ and to obtain efficient solutions with respect to both objectives.

Focusing on the tree case, the work in [10] addressed the BSSFP using a two-phase framework combined with a dynamic programming (DP) scheme. Both phases of this approach rely on a bottom-up DP scheme on rooted trees, where each node aggregates information from its children. A key ingredient of the approach in [10] is the use of Lagrangian relaxation to handle the choice of incident edges and the allocation of centers among subtrees. This approach crucially exploits the structural properties of trees. A natural question is whether this approach can be extended to more general graph classes. Cactus graphs constitute a strict generalization of trees, as they allow cycles while preserving a block structure in which any two cycles share at most one node. However, the presence of cycles destroys the subtree independence exploited by the DP and Lagrangian relaxation used in the tree case [10]. In a cactus graph, the decision of whether to include an edge on a cycle cannot be made locally, since it constrains the roles of multiple nodes on the same cycle and therefore couples several subproblems that would be independent on a tree. As a result, this approach cannot be directly extended to cactus graphs. On the other hand, in the mono-objective version, the polynomial time result of the maximum weight SSF problem from trees to cactus graphs [11] shows that cactus graphs remain solvable despite the presence of cycles. This result motivates the investigation of whether an efficient approach can be developed for the BSSFP on cactus graphs. In contrast, the bi-objective version on the cactus graph is more challenging, as the selection of edges and centers on a cycle may affect the trade-off between objectives in different ways.

In this paper, we show that the BSSFP can be solved in polynomial time on cactus graphs by presenting an exact algorithm that enumerates all efficient solutions. More precisely, we adopt the classical two-phase approach for bi-objective combinatorial optimization [14]. Phase I enumerates all supported efficient solutions obtained through weighted sum scalarization [12], while Phase II extends the search to non-supported efficient solutions using the $\varepsilon$-constraint method [8]. In both phases, the problem reduces to solving mono-objective subproblems on cactus graphs. The key technical contribution of this paper is to handle the dependencies induced by cycles by combining local DP states along each cycle through a min-plus product [2]. While the approach of [10] crucially depends on Lagrangian relaxation and preserves subtree independence, such a technique cannot be applied in the presence of cycles. Our idea is to exploit the block structure of cactus graphs and to separate the treatment

of tree components and cycle components. While the bottom-up DP recurrences can still handle tree components, cycle components require a different configuration that explicitly accounts for their global dependencies. For cycle components, we introduce an algebraic formulation based on the min-plus product that allows DP values computed at different nodes of a cycle to be combined consistently. Rather than relying on subtree independence, this formulation encodes all feasible local states along a cycle and aggregates them through the min-plus product, which exactly captures the interactions induced by the cycle. It is important to note that bi-objective combinatorial optimization problems are generally NP-hard, even when their mono-objective version is solvable in polynomial time, as illustrated for instance by the bi-objective spanning tree problem [6]. To the best of our knowledge, apart from the work in [10] studied on trees, polynomial time enumeration of the Pareto front is extremely rare; our result is one of the very few cases in which the Pareto front can be determined in polynomial time on a non-trivial graph class.

The remainder of this paper is organized as follows. Section 2 presents the structural decomposition of cactus graphs for our approach that is used throughout the paper. Section 3 describes the detailed algorithmic framework for solving the BSSFP on cactus graphs, which is based on a two-phase dynamic programming approach. Finally, Section 4 concludes the work and discusses possible directions for future research.

## 2 Structural decomposition of cactus graphs

To exploit the specific structure of cactus graphs in our algorithmic development, we describe a decomposition that represents the cactus graph as a rooted tree of simpler components. A node $v \in V$ is called a *cut node* if removing $v$ together with all its incident edges disconnects the graph. A node $v \in V$ is called a *pendant node* if it has degree one in $G$. A *block* of $G$ is a maximal connected subgraph that contains no cut nodes. Since $G$ is a cactus graph, every block of $G$ is either a single edge or a cycle. Following the approach introduced in [11], we construct a rooted tree structure, denoted by $\mathcal{T}$ and referred to as the $\mathcal{T}$*-tree*, whose nodes are pseudo-nodes representing the fundamental components of $G$. The construction relies on a standard decomposition based on a depth-first search (DFS) traversal of $G$. The pseudo-nodes of $\mathcal{T}$ are of two types:

- *singleton pseudo-nodes*, corresponding to cut nodes or pendant nodes of $G$;
- *cycle pseudo-nodes*, corresponding to blocks of $G$ that are cycles.

The $\mathcal{T}$-tree is built incrementally during the DFS as follows. Whenever a block $B$ of $G$ is identified together with its highest ancestor $w$ in the DFS tree, we distinguish two cases.

- If $B$ consists of a single edge $vw$, then two singleton pseudo-nodes corresponding to $v$ and $w$ are created in $\mathcal{T}$, and the pseudo-node associated with $w$ is set as the parent of the pseudo-node associated with $v$.
- If $B$ is a cycle, then a cycle pseudo-node representing $B$ is created, together with a singleton pseudo-node corresponding to its root node $w$. The singleton pseudo-node

$w$ is set as the parent of the cycle pseudo-node $B$. Moreover, for every cut node $v$ belonging to the cycle $B$, the pseudo-node $B$ is set as the parent of the singleton pseudo-node corresponding to $v$.

For any singleton pseudo-node $w$ that has exactly one child and whose unique child is a cycle pseudo-node $B$, the node $w$ is removed from $\mathcal{T}$ and $B$ is directly linked to the parent of $w$. The resulting $\mathcal{T}$-tree is a rooted tree that represents the global structure of the cactus graph. In particular, each node of the original graph $G$ is assigned to exactly one pseudo-node of $\mathcal{T}$. In particular, cut nodes are represented by singleton pseudo-nodes, while cycle pseudo-nodes represent blocks and do not duplicate graph nodes. For each pseudo-node $N \in \mathcal{T}$, we associate a representative node $u_N \in V$, defined as $u_N = N$ if $N$ is a singleton pseudo-node, and as the root node of the corresponding cycle if $N$ is a cycle pseudo-node. With a careful implementation, the $\mathcal{T}$-tree can be constructed in linear time $O(|V| + |E|)$. Based on this decomposition, we now present our algorithm for solving the BSSFP in the next Section.

# 3 Algorithms for solving the BSSFP on cactus graphs

## 3.1 Algorithm construction

This section presents the overall algorithmic framework for solving the BSSFP on cactus graphs. While all efficient solutions of multi-objective linear programming problems are supported, due to the convexity of the feasible set in the objective space [13], this property does not extend to discrete optimization problems. In the BSSFP, both the total weight and the number of centers take discrete values, which implies that the Pareto front is finite and generally non-convex. As a result, the Pareto front may contain both supported and non-supported efficient solutions. In general, non-supported efficient solutions are more challenging to identify, since they cannot be obtained by weighted sum scalarizations. To this end, we adopt the classical two-phase method for bi-objective combinatorial optimization [14], which has been widely used to systematically generate supported solutions in the first phase and non-supported solutions in the second phase. The approach proceeds as follows.

- *Phase I (supported efficient solutions).* Phase I enumerates all supported efficient solutions of the BSSFP by solving a sequence of weighted sum scalarized subproblems. These subproblems are generated through a dichotomic search on the scalarization weights and are solved exactly by DP on a decomposition of the cactus graph into singleton and cycle pseudo-nodes. We now deal with the weighted sum subproblem of the BSSFP. On trees, this subproblem can be solved by DP schemes that rely on the independence of subtrees and can therefore be handled by standard bottom-up recurrences. However, on cactus graphs, cycle pseudo-nodes introduce circular dependencies between nodes on the same cycle, which invalidate subtree-based recurrences. Hence, we process each cycle as an ordered sequence of nodes and aggregate the corresponding DP states using a min-plus product, which allows all feasible edge selections on the cycle to be handled exactly.
- *Phase II (non-supported efficient solutions).* Phase II aims to enumerate all non-supported efficient solutions of the BSSFP by exploiting the ordered list of supported

solutions obtained in Phase I. For each pair of consecutive supported solutions, we solve a sequence of $\varepsilon$-constraint subproblems that minimize the total weight subject to an upper bound on the number of centers, in order to obtain all non-supported efficient solutions lying between the two supported endpoints.
We now focus on the $\varepsilon$-constraint subproblem of the BSSFP. On trees, such constrained subproblems can be handled by DP schemes that distribute the center bound locally across subtrees. On cactus graphs, however, the presence of cycles couples local decisions along the cycle, so the center bound cannot be enforced independently. The main challenge in computing non-supported efficient solutions is therefore to handle the interaction between cycle dependencies and the center constraint. Thus, we propagate the DP states along each cycle using a min-plus product, which ensures global consistency of the center constraint and allows all feasible configurations on the cycle to be considered.

Finally, we show that our algorithm enumerates the Pareto front of the BSSFP on a cactus graph in $O(|E|^2|V|)$ time. We now start with the first phase.

## 3.2 Phase I: Enumeration of supported efficient solutions

This section presents Phase I of our approach, whose goal is to enumerate all supported efficient solutions of the BSSFP on a cactus graph $G$. The general idea is to solve a sequence of mono-objective problems obtained by weighted sum scalarization. Supported efficient solutions are enumerated by applying a dichotomic search that recursively solves these scalarized subproblems. The weighted sum subproblem is then solved by DP on the decomposition of the cactus graph into singleton pseudo-nodes and cycle pseudo-nodes. For a singleton pseudo-node, the DP values are obtained by combining the values computed for its children. However, for a cycle pseudo-node, the nodes of the cycle are mutually dependent through the cycle edges, which breaks the independence property required by standard bottom-up recurrences. To resolve this issue, we process each cycle as an ordered sequence of nodes. Since the choice of an edge on the cycle directly constrains the choices of adjacent edges, the DP states along the cycle cannot be combined independently. We therefore use the min-plus product to aggregate the DP values associated with consecutive nodes, which propagates all admissible state transitions along the cycle. In this way, all feasible edge selections on the cycle are handled, and cycle pseudo-nodes are treated exactly in the overall DP.

We now recall the formal definition of supported solutions used throughout this section. Let $G = (V, E)$ be a cactus graph where each edge $ij \in E$ is associated with a non-negative weight $w_{ij}$. A solution of BSSFP is defined as *supported* if it is an optimal solution of the weighted sum problem $\mathcal{W}_\alpha$ [12]:

$$\min_{(f_1,f_2)\in\mathcal{F}} \mathcal{W}_\alpha(f_1, f_2) = (1-\alpha)f_1 + \alpha f_2,$$

where $\alpha \in [0, 1]$ is a scalarization coefficient. To identify the set of supported solutions, we implement Procedure FIND_SUPPORTED, an iterative scheme based on the two-phase method introduced in [7]. This algorithm identifies all the supported efficient solutions. The process initiates at the boundaries by solving $\mathcal{W}_0 = \min f_1$ and

$\mathcal{W}_1 = \min f_2$, yielding the boundary efficient points $(f_1^0, f_2^0)$ and $(f_1^1, f_2^1)$. Notably, a feasible solution where every node serves as an isolated node constitutes a SSF; thus, the minimization of $f_1$ leads to the trivial solution $(0, |V|)$. Given that these endpoints are efficient, they satisfy the conditions $f_1^0 < f_1^1$ and $f_2^0 > f_2^1$. These points define the initial search interval. For any segment defined by two current solutions $[(f_1^i, f_2^i), (f_1^j, f_2^j)]$, the procedure investigates the existence of a new supported solution strictly between them. The search weight $\alpha$ is chosen such that both endpoints are indifferent with respect to the weighted sum objective. Solving $(1-\alpha)f_1^i + \alpha f_2^i = (1-\alpha)f_1^j + \alpha f_2^j$ yields $\alpha = \frac{f_1^j - f_1^i}{(f_1^j - f_1^i) + (f_2^i - f_2^j)}$. Applying $\alpha$ to $\mathcal{W}_\alpha$ produces a new candidate solution $(f_1^m, f_2^m)$. If this candidate coincides with either endpoint, the segment contains no further supported points. Otherwise, the new solution is incorporated into the ordered set $\mathcal{S}$, and the search is recursively applied to the two newly formed subsegments. This process terminates when no segment remains to be explored.

**Algorithm 1** Enumerating supported efficient solutions via $\mathcal{W}_\alpha$

**Input:** A weighted cactus graph $G = (V, E, w)$.
**Output:** Ordered set $\mathcal{S}$ of supported efficient solutions.

**procedure** *FIND_SUPPORTED()*
  Solve $\mathcal{W}_0$ to obtain $(f_1^0, f_2^0)$
  Solve $\mathcal{W}_1$ to obtain $(f_1^1, f_2^1)$
  $\mathcal{S} \leftarrow \{(f_1^0, f_2^0), (f_1^1, f_2^1)\}$ ordered by increasing $f_1$
  $L \leftarrow \{[(f_1^0, f_2^0), (f_1^1, f_2^1)]\}$
  **while** $L \neq \emptyset$ **do**
    Select and remove $[(f_1^i, f_2^i), (f_1^j, f_2^j)]$ from $L$
    **if** $f_1^i \neq f_1^j$ **and** $f_2^i \neq f_2^j$ **then**
      $\alpha \leftarrow \dfrac{f_1^j - f_1^i}{(f_1^j - f_1^i) + (f_2^i - f_2^j)}$
      Solve $\mathcal{W}_\alpha$ to obtain $(f_1^m, f_2^m)$
      **if** $(f_1^m, f_2^m) \neq (f_1^i, f_2^i)$ **and** $(f_1^m, f_2^m) \neq (f_1^j, f_2^j)$ **then**
        Insert $(f_1^m, f_2^m)$ into $\mathcal{S}$ preserving the order of $f_1$
        $L \leftarrow L \cup \{[(f_1^i, f_2^i), (f_1^m, f_2^m)], [(f_1^m, f_2^m), (f_1^j, f_2^j)]\}$
      **end if**
    **end if**
  **end while**
  **return** $\mathcal{S}$
**end procedure**

We now propose a DP algorithm that solves $\mathcal{W}_\alpha$ exactly in linear time on a cactus. Let $S_N$ denote the subgraph of $G$ induced by the set of nodes corresponding to the subtree of the $\mathcal{T}$-tree rooted at a pseudo-node $N$. For a fixed value of $\alpha \in [0, 1]$ and for each pseudo-node $N$ of the $\mathcal{T}$-tree, we define three values associated with the representative node $u_N$.

- *Center case* $\Phi_\alpha(N)$: the optimal value of $\mathcal{W}_\alpha$ over all SSFs of $S_N$, given that $u_N$ is the center of a star and that this star contains at least one edge incident to $u_N$.
- *Leaf case* $\Psi_\alpha(N)$: the optimal value of $\mathcal{W}_\alpha$ over all SSFs of $S_N$, given that $u_N$ is a leaf of a star.
- *Isolated case* $\Omega_\alpha(N)$: the optimal value of $\mathcal{W}_\alpha$ over all SSFs of $S_N$, given that $u_N$ is an isolated node.

When the cactus reduces to a tree, the weighted sum problem of BSSFP can be solved by a standard bottom-up DP scheme [10]. The following lemma extends this recurrence to cactus graphs by processing a singleton pseudo-node through its singleton and cycle pseudo-node children in the decomposition tree $\mathcal{T}$.

**Lemma 1** Let $N$ be a singleton pseudo-node of the decomposition tree $\mathcal{T}$. Let $CS(N)$ be the set of singleton pseudo-node children of $N$ and $CC(N)$ the set of cycle pseudo-node children of $N$ in $\mathcal{T}$. For each $v \in CS(N)$, let $w_{Nv}$ be the weight of the bridge edge $Nv$ in $G$. We define

$$\Delta(v) = \min\{\Phi(v), \Psi(v), \Omega(v) + (1-\alpha)w_{Nv}\},$$
$$\delta(v) = \min\{\Phi(v), \Psi(v), \Omega(v)\}.$$

Then the following relations hold:

$$\Omega(N) = \alpha + \sum_{B \in CC(N)} \Omega(B) + \sum_{v \in CS(N)} \delta(v),$$

$$\Phi(N) = \alpha + \min\Big\{ \min_{B \in CC(N)} \big(\Phi(B) + \sum_{B' \in CC(N)\setminus\{B\}} \min\{\Phi(B'), \Omega(B')\} + \sum_{v \in CS(N)} \Delta(v)\big),$$
$$\sum_{B \in CC(N)} \Omega(B) + \sum_{v \in CS(N)} \Delta(v) + \min_{v \in CS(N)} \big(\Omega(v) + (1-\alpha)w_{Nv} - \Delta(v)\big)\Big\},$$

$$\Psi(N) = \min\Big\{ \min_{B \in CC(N)} \big(\Psi(B) + \sum_{B' \in CC(N)\setminus\{B\}} \Omega(B') + \sum_{v \in CS(N)} \delta(v)\big),$$
$$\min_{v \in CS(N)} \big((1-\alpha)w_{Nv} + \Phi(v) + \sum_{B \in CC(N)} \Omega(B) + \sum_{x \in CS(N)\setminus\{v\}} \delta(x)\big)\Big\}.$$

*Proof* We first justify the validity of the formula for $\Omega(N)$. A minimum weighted sum SSF in $S_N$ where $N$ is isolated contains no edge incident to $N$. Hence, for every cycle pseudo-node $B \in CC(N)$, the node $N$ is isolated in $S_B$ and the value is $\Omega(B)$. For every singleton pseudo-node $v \in CS(N)$, no restriction is imposed on the role of $v$, and the minimum weighted sum value in $S_v$ is $\delta(v)$.

We next consider $\Psi(N)$, which accounts for two possible cases. Either $N$ is a leaf of a star centered in the subgraph $S_B$ for some cycle pseudo-node $B \in CC(N)$. In this case, the value in $S_B$ is $\Psi(B)$, the node $N$ is isolated in every other subgraph $S_{B'}$ with $B' \in CC(N) \setminus \{B\}$, yielding the value $\Omega(B')$, and each singleton pseudo-node $v \in CS(N)$ induces the value $\delta(v)$. Otherwise, $N$ is a leaf of a star centered at a singleton pseudo-node $v \in CS(N)$. Then the edge $Nv$ must belong to the star forest, which induces the term $(1-\alpha)w_{Nv}$, and the value in $S_v$ is $\Phi(v)$. All cycle pseudo-nodes $B \in CC(N)$ induce the value $\Omega(B)$, and all other singleton pseudo-nodes $x \in CS(N) \setminus \{v\}$ induce the value $\delta(x)$.

We now turn to the expression for $\Phi(N)$. There are two possible cases for a star centered at $N$. Either there exists a cycle pseudo-node $B \in CC(N)$ such that the star contains an

edge incident to $N$ in the subgraph $S_B$. In this case, the star forest in $S_B$ has value $\Phi(B)$, every other cycle pseudo-node $B' \in CC(N) \setminus \{B\}$ induces the value $\min\{\Phi(B'), \Omega(B')\}$, and each singleton pseudo-node $v \in CS(N)$ induces the value $\Delta(v)$. Otherwise, the star centered at $N$ does not intersect any cycle pseudo-node. Then all cycle pseudo-nodes $B \in CC(N)$ induce the value $\Omega(B)$, and each singleton pseudo-node $v \in CS(N)$ induces the value $\Delta(v)$. In order to satisfy the center condition, there must exist at least one singleton pseudo-node $v \in CS(N)$ such that the edge $Nv$ belongs to the star forest. In this case, the star forest corresponding to $\Phi(N)$ necessarily contains the edge $Nv$ and the subset of edges associated with $\Omega(v)$ for $v = \arg\min_{v \in CS(N)} \big(\Omega(v) + (1-\alpha)w_{Nv} - \Delta(v)\big)$. The two cases are considered respectively in the first and second parts of the formula for $\Phi(N)$, and the fixed $\alpha$ accounts for selecting $N$ as a center. □

We now consider the case where $N$ is a cycle pseudo-node of the decomposition tree $\mathcal{T}$. Let $C = (1, 2, \ldots, n, 1)$ be the corresponding cycle, where $n = u_N$ is the root node of $N$. Its edges are $e_i = (i, i+1)$ for $i = 1, \ldots, n-1$ and $e_n = (n, 1)$, with weights $w_{e_i}$. For each $i \in C \setminus \{n\}$, if $i$ is a singleton pseudo-node, the values $\Phi(i)$, $\Psi(i)$, and $\Omega(i)$ are computed following the bottom-up formulation in Lemma 1. Otherwise, $i$ has no descendants, and we set $\Omega(i) = 0$, $\Psi(i) = 0$, $\Phi(i) = +\infty$, which are exactly the values of the weighted sum objective $\mathcal{W}_\alpha$ for a single node in the isolated, leaf, and center states, respectively.

To compute the values associated with $N$, we remove the edge $e_n = (n, 1)$ and consider the resulting graph $H$, which consists of the path on nodes $1, \ldots, n$, the edges $e_1, \ldots, e_{n-1}$, and all descendant subgraphs attached to these nodes. Each node $i$ is assigned a state $s(i) \in \{0, 1, 2\}$, corresponding respectively to an isolated node, a leaf, or a center. For $i = 1, \ldots, n-1$, we associate with the edge $e_i = (i, i+1)$ a transition matrix $T_i \in (\mathbb{R} \cup \{+\infty\})^{3\times 3}$. For $q \in \{0, 1, 2\}$, let

$$D_{i+1}(q) = \begin{cases} \Omega(i+1), & q = 0, \\ \Psi(i+1), & q = 1, \\ \Phi(i+1), & q = 2, \end{cases}$$

denote the optimal contribution of the descendant subgraph attached to node $i+1$ when it is in state $q$. The value $T_i[p, q]$ represents the contribution to the weighted sum objective $\mathcal{W}_\alpha$ arising from the descendant subgraph of node $i+1$, together with the possible selection of the cycle edge $e_i$, under the condition that $s(i) = p$ and $s(i+1) = q$. It is defined by

$$T_i[p, q] = \begin{cases} D_{i+1}(q) + (1-\alpha)w_{e_i}, & \text{if } (p, q) \in \{(1, 2), (2, 1)\}, \\ D_{i+1}(q), & \text{otherwise.} \end{cases}$$

If no feasible spanning star forest of $H$ satisfies the conditions $s(i) = p$ and $s(i+1) = q$, we set $T_i[p, q] = +\infty$. By construction, $T_i[p, q]$ includes the contribution of the descendant subgraph attached to node $i+1$, but does not include any contribution from the descendant subgraph of node $i$. This separation ensures that each descendant subgraph is counted exactly once. For any spanning star forest $S$ of $H$ inducing a

state sequence $(s(1),\dots,s(n))$, the weighted sum objective decomposes additively as $U_1\big(s(1)\big)+\sum_{i=1}^{n-1} T_i\big[s(i),s(i+1)\big]$, where $U_1(0)=\Omega(1),\ U_1(1)=\Psi(1),\ U_1(2)=\Phi(1)$.

To optimize over all state sequences with fixed boundary states, we use the min-plus matrix product $\otimes$, defined by $(A\otimes B)_{ij}=\min_{k\in\{0,1,2\}}\big(A_{ik}+B_{kj}\big)$. Let $C=T_1\otimes T_2\otimes\cdots\otimes T_{n-1}\in(\mathbb{R}\cup\{+\infty\})^{3\times 3}$. For $p,q\in\{0,1,2\}$, we denote by $C_{pq}$ the element of the matrix $C$ located in row $p$ and column $q$.

**Claim 1** For any $p,q\in\{0,1,2\}$, the value $C_{pq}$ equals

$$\min\Big\{\sum_{i=1}^{n-1} T_i[s(i),s(i+1)]\ \Big|\ (s(1),\dots,s(n))\in\{0,1,2\}^n,\ s(1)=p,\ s(n)=q\Big\}.$$

*Proof* For $k=1,\dots,n-1$, define the partial product $C^{(k)}=T_1\otimes\cdots\otimes T_k$. We prove by induction on $k$ that, for every $p,r\in\{0,1,2\}$,

$$C^{(k)}_{pr}=\min\Big\{\sum_{i=1}^{k} T_i[s(i),s(i+1)]\ \Big|\ (s(1),\dots,s(k+1))\in\{0,1,2\}^{k+1},\ s(1)=p,\ s(k+1)=r\Big\}.$$

For $k=1$, we have $C^{(1)}=T_1$, hence $C^{(1)}_{pr}=T_1[p,r]$, which coincides with the above minimum over the unique length-2 state sequence satisfying $s(1)=p$ and $s(2)=r$. Assume the statement holds for some $k\in\{1,\dots,n-2\}$. Since $C^{(k+1)}=C^{(k)}\otimes T_{k+1}$, for any $p,q\in\{0,1,2\}$ we obtain from the definition of $\otimes$ that $C^{(k+1)}_{pq}=\min_{r\in\{0,1,2\}}\big(C^{(k)}_{pr}+T_{k+1}[r,q]\big)$. By the induction hypothesis, for each fixed $r$ the term $C^{(k)}_{pr}$ is the minimum of $\sum_{i=1}^{k} T_i[s(i),s(i+1)]$ over all state sequences with $s(1)=p$ and $s(k+1)=r$. Adding $T_{k+1}[r,q]$ extends any such sequence by one transition to a sequence of length $k+2$ with endpoint states $s(1)=p$ and $s(k+2)=q$. Taking the minimum over all $r\in\{0,1,2\}$ therefore yields

$$C^{(k+1)}_{pq}=\min\Big\{\sum_{i=1}^{k+1} T_i[s(i),s(i+1)]\ \Big|\ (s(1),\dots,s(k+2))\in\{0,1,2\}^{k+2},\ s(1)=p,\ s(k+2)=q\Big\},$$

which completes the induction. Finally, taking $k=n-1$ gives the claimed expression for $C_{pq}=C^{(n-1)}_{pq}$. □

The edge $e_n=(n,1)$ and the descendant subgraph attached to node 1 are then considered. By enumerating all feasible choices for including or excluding $e_n$ and combining them with the states of nodes 1 and $n$, the values $\Omega(N)$, $\Psi(N)$, and $\Phi(N)$ are obtained. This completes the computation for the cycle pseudo-node $N$.

**Lemma 2** Let $N$ be a cycle pseudo-node corresponding to the cycle $C=(1,2,\dots,n,1)$. Then the values associated with $N$ are given by:

$$\Omega(N)=\alpha+\min\Big\{C_{00}+\Omega(1),\ C_{10}+\Psi(1),\ C_{20}+\Phi(1)\Big\},$$

$$\Psi(N)=\min\Big\{C_{01}+\Omega(1),\ C_{11}+\Psi(1),\ C_{21}+\Phi(1),\ C_{20}+(1-\alpha)w_{e_n}+\Phi(1)\Big\},$$

$$\Phi(N)=\alpha+\min\Big\{C_{02}+\Omega(1),\ C_{12}+\Psi(1),\ C_{22}+\Phi(1),$$
$$C_{01}+(1-\alpha)w_{e_n}+\Omega(1),\ C_{02}+(1-\alpha)w_{e_n}+\Omega(1)\Big\}.$$

*Proof* By the construction of the transition matrices $T_i$ and the DP procedure described above, for each $p, q \in \{0, 1, 2\}$, the value $C_{pq}$ represents the minimum value of the weighted sum objective over all SSFs of $H$ such that node 1 is in state $p$ and node $n$ is in state $q$ (with 0/1/2 standing respectively for isolated/leaf/center). We now take into account the edge $e_n = (n, 1)$ and the descendant subgraph of node 1.
- *Isolated case.* If node $n$ is isolated in the star forest, then the edge $e_n$ cannot be selected, and node $n$ must be in state 0 in $H$. Combining the corresponding solutions on $H$ with the three possible states of node 1 yields the values $C_{00} + \Omega(1)$, $C_{10} + \Psi(1)$, and $C_{20} + \Phi(1)$.
- *Leaf case.* If node $n$ is a leaf in the star forest, then either it is already a leaf in $H$ (i.e., $n$ is in state 1 in $H$), or it becomes a leaf by selecting the edge $e_n$ and attaching it to a center at node 1. These two possibilities correspond respectively to the terms $C_{01} + \Omega(1)$, $C_{11} + \Psi(1)$, $C_{21} + \Phi(1)$, and $C_{20} + (1 - \alpha)w_{e_n} + \Phi(1)$.
- *Center case.* If node $n$ is a center in the star forest, then either it is already a center in $H$ (i.e., $n$ is in state 2 in $H$), or it becomes a center by selecting the edge $e_n$. In the first situation, the value is obtained by combining the terms $C_{02} + \Omega(1)$, $C_{12} + \Psi(1)$, and $C_{22} + \Phi(1)$. In the second situation, node $n$ is isolated in $H$ (state 0) and the edge $e_n$ is selected, which yields the terms $C_{01} + (1 - \alpha)w_{e_n} + \Omega(1)$ and $C_{02} + (1 - \alpha)w_{e_n} + \Omega(1)$. □

We now state Procedure SOLVE_Weighted_Sum for solving the weighted sum subproblem $\mathcal{W}_\alpha$. The procedure first constructs the decomposition tree $\mathcal{T}$ of the cactus graph and then processes its pseudo-nodes in a bottom-up order. For each pseudo-node $N$, the DP values $\Phi(N)$, $\Psi(N)$, and $\Omega(N)$ are computed. If $N$ is a singleton pseudo-node, these values are obtained using the recurrences of Lemma 1, otherwise, when $N$ corresponds to a cycle pseudo-node, they are computed using the procedure described in Lemma 2. After all pseudo-nodes have been processed, the algorithm determines the optimal solution by taking the minimum among $\Phi(r)$, $\Psi(r)$, and $\Omega(r)$ at the root pseudo-node $r$, and returns the corresponding SSF.

**Algorithm 2** Solving the weighted sum problem $\mathcal{W}_\alpha$

**Input:** A weighted cactus graph $G = (V, E, w)$ and a coefficient $\alpha \in [0, 1]$.
**Output:** A SSF associate with the optimal solution for $\mathcal{W}_\alpha$.

**procedure** *SOLVE_Weighted_Sum()*
  Construct the decomposition tree $\mathcal{T}$ of $G$ as described in Section 2
  Let $r$ be the root pseudo-node of $\mathcal{T}$
  **for all** pseudo-nodes $N$ of $\mathcal{T}$ in bottom-up order **do**
    **if** $N$ is a singleton pseudo-node **then**
      Compute $\Phi(N)$, $\Psi(N)$, and $\Omega(N)$ using the recurrences in Lemma 1
    **else**
      Compute $\Phi(N)$, $\Psi(N)$, and $\Omega(N)$ using the processing procedure for cycle pseudo-node described above and Lemma 2.
    **end if**
    Store the selected edges associated with the computed values
  **end for**
  $S \leftarrow \min\{\Phi(r), \Psi(r), \Omega(r)\}$
  **return** the SSF corresponding to $S$
**end procedure**

**Theorem 1** *When $G$ is a cactus graph, Algorithm 2 solves the weighted sum problem $\mathcal{W}_\alpha$ in $O(|E|)$ time.*

*Proof* By Lemma 1 and Lemma 2, the evaluations of $\Phi(N)$, $\Psi(N)$, and $\Omega(N)$ in Step 2 are correct for singleton and cycle pseudo-nodes, respectively. Therefore, the algorithm returns an optimal solution for $\mathcal{W}_\alpha$. Step 1 constructs the decomposition tree $\mathcal{T}$ in $O(|E|)$ time. The computation time of the values $\Phi(B)$, $\Psi(B)$, and $\Omega(B)$ associated with a singleton pseudo-node $B$ of $\mathcal{T}$ is $O(|CS(B)| + |CC(B)|)$, and the computation time associated with a cycle pseudo-node $B$ corresponding to a cycle $C$ is $O(|C|)$. Since the blocks of a cactus graph are edge-disjoint, we get that the time complexity of the algorithm is $O(|E|)$. □

### 3.3 Phase II: Enumeration of non-supported efficient solutions

This section presents Phase II of our approach, whose goal is to enumerate all non-supported efficient solutions of the BSSFP on a cactus graph $G$. While Phase I identifies the supported solutions by solving weighted sum scalarizations, Phase II explores the remaining efficient solutions that cannot be obtained by the weighted sum method. The general idea is to exploit the ordered list of supported solutions obtained in Phase I and to search systematically between each pair of consecutive supported solutions. For each pair, we obtain all non-supported efficient solutions between these two endpoints by solving a sequence of $\varepsilon$-constraint subproblems, in which the total weight is minimized with a bound on the number of centers. This bound depends on the information from the pair of consecutive supported solutions. Each $\varepsilon$-constraint subproblem is solved by DP on the decomposition of the cactus graph into singleton and cycle pseudo-nodes, as described in Section 2. The central idea in this phase is to enforce the bound on the number of centers while combining partial solutions along the decomposition. For singleton pseudo-nodes, the center constraint can be handled locally by assigning the available number of centers to the children. For cycle pseudo-nodes, however, local decisions are coupled by the cycle edges, and the number of centers must be controlled consistently along the cycle. To address this difficulty, we again rely on the min-plus product to propagate DP information along the cycle while preserving feasibility with respect to the center constraint.

Phase II proceeds by examining the solution space between consecutive supported efficient solutions. In particular, each such pair $(f_1^i, f_2^i)$ and $(f_1^j, f_2^j)$ defines a region that may contain non-supported efficient solutions. Without loss of generality, we assume that $f_2^i < f_2^j$. To obtain these solutions, we utilize the $\epsilon$-constraint scalarization technique [8]. In the context of the BSSFP on a cactus graph $G = (V, E)$, the $\varepsilon$-constraint approach consists of minimizing the total weight of the SSF while treating the number of centers as a constraint. For a given integer $k$, the $\epsilon$-constraint subproblem is defined as

$$(\mathcal{P}_k) \quad \min\{f_1 : (f_1, f_2) \in \mathcal{F}, f_2 \leq k\}.$$

The search for non-supported solutions is detailed in Procedure FIND_NONSUPPORTED (Algorithm 3). Given two consecutive supported solutions,

the procedure initializes a search bound $k = f_2^j - 1$ and an empty set $NS$ to store the discovered points. In each iteration, the subproblem $(\mathcal{P}_k)$ is solved to obtain an objective vector $(f_1^u, f_2^u)$. If this pair differs from the two endpoints, it corresponds to a non-supported efficient solution and is added to $NS$. The bound is then updated to $f_2^u - 1$, thereby restricting the search to the remaining part of the objective space. This iterative refinement continues as long as there exists an integer $k$ strictly between the bounds $f_2^i$ and $f_2^j$. Since the number of centers $f_2$ is an integer value, this process is guaranteed to terminate and capture all non-supported efficient solutions between two endpoints.

**Algorithm 3** Enumerating non-supported efficient solutions

**Input:** A weighted cactus graph $G = (V, E, w)$ and two consecutive supported efficient solutions $(f_1^i, f_2^i)$, $(f_1^j, f_2^j)$ with $f_2^i < f_2^j$.

**Output:** Set $NS$ of non-supported efficient solutions between the two endpoints.

```
procedure FIND_NONSUPPORTED()
    NS ← ∅
    k ← f_2^j − 1
    while k ≥ f_2^i + 1 do
        Solve the constrained problem (P_k) to obtain (f_1^u, f_2^u)
        NS ← NS ∪ {(f_1^u, f_2^u)}
        k ← f_2^u − 1
    end while
    return NS
end procedure
```

We solve the $\epsilon$-constraint subproblem $(\mathcal{P}_k)$ by DP on the decomposition tree $\mathcal{T}$. In the tree case, $(\mathcal{P}_k)$ can be solved by a bottom-up DP recurrence on the rooted tree, as established in [10]. As a special instance of cactus graphs, Lemma 3 extends the recurrence to cactus graphs by exploiting the $\mathcal{T}$-tree decomposition and by handling singleton and cycle pseudo-nodes separately. For each pseudo-node $N$ of $\mathcal{T}$, let $S_N$ be the subgraph of $G$ induced by the nodes corresponding to the subtree of $\mathcal{T}$ rooted at $N$, and let $u_N$ be the representative node of $N$. For every integer $\ell \geq 0$, we define:

- $\Phi(N, \ell)$: minimum total weight of a SSF in $S_N$ using exactly $\ell$ centers, where $u_N$ is a center with at least one child.
- $\Psi(N, \ell)$: minimum total weight of a SSF in $S_N$ using exactly $\ell$ centers, where $u_N$ is a leaf.
- $\Omega(N, \ell)$: minimum total weight of a SSF in $S_N$ using exactly $\ell$ centers, where $u_N$ is isolated.

**Lemma 3** Let $N$ be a singleton pseudo-node of the $\mathcal{T}$-tree. Let $CS(N)$ be the set of singleton pseudo-node children of $N$ and $CC(N)$ the set of cycle pseudo-node children of $N$. For each $v \in CS(N)$, let $w_{Nv}$ be the weight of the bridge edge $Nv$ in $G$. For every $v \in CS(N)$ and every integer $p \geq 0$, let

$$\Delta(v,p) = \min\big\{\Phi(v,p),\ \Psi(v,p),\ \Omega(v,p) + w_{Nv}\big\},\quad \delta(v,p) = \min\big\{\Phi(v,p),\ \Psi(v,p),\ \Omega(v,p)\big\}.$$

Let $s = \sum_{v \in CS(N)} p_v + \sum_{B \in CC(N)} q_B$. Then for every $\ell \geq 0$,

$$\Phi(N,\ell) = \min\Big\{ \min_{\substack{I \subseteq CC(N) \\ I \neq \varnothing}} \Big( \sum_{B \in I} \Phi(B, q_B) + \sum_{B \notin I} \Omega(B, q_B) + \sum_{v} \Delta(v, p_v) \Big| \, s = \ell - 1 + |I| \Big),$$
$$\min\Big( \sum_{B} \Omega(B, q_B) + \sum_{v} \Delta(v, p_v) + \min_{v} \big(\Omega(v, p_v) + w_{Nv} - \Delta(v, p_v)\big) \Big| \, s = \ell - 1 \Big) \Big\},$$

$$\Psi(N,\ell) = \min\Big\{ \min_{B} \Big( \Psi(B, q_B) + \sum_{B' \neq B} \Omega(B', q_{B'}) + \sum_{v} \delta(v, p_v) \Big),$$
$$\min_{v} \Big( \Phi(v, p_v) + w_{Nv} + \sum_{B} \Omega(B, q_B) + \sum_{x \neq v} \delta(x, p_x) \Big) \Big| \, s = \ell \Big\},$$

$$\Omega(N,\ell) = \min\Big\{ \sum_{v} \delta(v, p_v) + \sum_{B} \Omega(B, q_B) \Big| \, s = \ell - 1 \Big\}.$$

*Proof* The proof follows the same case analysis as in Lemma 1, but now we must keep track of the exact number of centers. Note that when $u_N$ is isolated, it has no incident edge, then each child can be in any feasible state: singleton pseudo-node children give $\delta(v, p_v)$, cycle pseudo-node children give $\Omega(B, q_B)$ and the total number of centers in the children must be exactly $\ell - 1$. When $u_N$ is a leaf, there are two possibilities.

- The center of $u_N$ lies in a cycle child $B$. Then $B$ is in state $\Psi(B, q_B)$, all other cycles are $\Omega$, and all singleton pseudo-node children are $\delta(v, p_v)$.
- The center of $u_N$ is a singleton child $v$. Then we must take edge $Nv$ (weight $w_{Nv}$), $v$ is a center ($\Phi(v, p_v)$), all cycles are $\Omega$, and the other singleton pseudo-node children are $\delta(x, p_x)$.

In both cases, the total number of centers in the children is $\ell$. When $u_N$ is a center, it uses one center. Two cases are possible:

- *$u_N$ is connected to no cycle child.* Then all cycles are in state $\Omega(B, q_B)$. singleton pseudo-node children contribute $\Delta(v, p_v)$. To ensure that $u_N$ has at least one incident edge, we must connect it to some singleton child $v$. The extra value for forcing this connection is $\min_v(\Omega(v, p_v) + w_{Nv} - \Delta(v, p_v))$. The total number of centers in the children is $\ell - 1$.
- *$u_N$ is connected to at least one cycle child.* Let $I \subseteq CC(N)$ (with $I \neq \emptyset$) be the set of cycles to which $u_N$ is connected. For $B \in I$, the value $\Phi(B, q_B)$ is used, and $q_B$ already accounts for $u_N$ as a center. For $B \notin I$, the value $\Omega(B, q_B)$ applies. The singleton pseudo-node children contribute $\Delta(v, p_v)$. Since each cycle in $I$ already counts $u_N$ as a center, the total number of centers among the children must be $\ell - 1 + |I|$.

Since each child has only a constant number of possible states, evaluating these recurrences takes time linear in the number of children of $N$. □

Let $N$ be a cycle pseudo-node corresponding to the cycle $C = (1, 2, \ldots, n, 1)$, where $n = u_N$ is the root node. As in Phase I, we first remove the edge $e_n = (n, 1)$ and consider the resulting graph $H$, which consists of the path on nodes $1, \ldots, n$,

the edges $e_1, \ldots, e_{n-1}$, and all descendant subgraphs attached to these nodes. This transformation breaks the cycle and allows a sequential dynamic programming along the path. We use the same state encoding as in Phase I: state 0 means that a node is isolated, state 1 means that it is a leaf, and state 2 means that it is a center. We recall that the number of centers is counted as the number of star components; in particular, an isolated node contributes one center.

For every node $i \in C \setminus \{n\}$ that has no descendants in $\mathcal{T}$, the descendant subgraph of $i$ reduces to the single node $i$. Hence the only feasible configuration is that $i$ is isolated, and we set $\Omega(i,1) = 0$, $\Omega(i,\ell) = +\infty$ for $\ell \neq 1$, and $\Psi(i,\ell) = \Phi(i,\ell) = +\infty$ for all $\ell \geq 0$. To aggregate the contributions along the path $1, \ldots, n$, we define local transition arrays that describe how consecutive nodes can be combined. For each index $i = 1, \ldots, n-1$ and each state pair $(p,q) \in \{0,1,2\}^2$, we define a transition array $T_i[p,q,\cdot] : \mathbb{N}_{\geq 0} \to \mathbb{R} \cup \{+\infty\}$. For $m \geq 0$, the value $T_i[p,q,m]$ is the minimum total weight contributed by the local decision on the edge $e_i = (i, i+1)$, adding $w_{e_i}$ if this edge is selected, together with the descendant subgraph of node $i+1$, under the conditions that $s(i) = p$, $s(i+1) = q$, and that exactly $m$ centers are used in the descendant part of node $i+1$. If no spanning star forest satisfies these conditions, we set $T_i[p,q,m] = +\infty$. These transition arrays have constant state dimension and encode all feasible local configurations between nodes $i$ and $i+1$. To combine them along the path, we extend the min-plus product so that it also tracks the number of centers by

$$(A \otimes B)[p,q,\ell] = \min_{\substack{r \in \{0,1,2\} \\ m_1 + m_2 = \ell}} \big(A[p,r,m_1] + B[r,q,m_2]\big).$$

We then define $C = T_1 \otimes T_2 \otimes \cdots \otimes T_{n-1}$. By construction, for any $p, q \in \{0,1,2\}$ and any $\ell \geq 0$, the value $C[p,q,\ell]$ is the minimum total weight contributed by the edges $e_1, \ldots, e_{n-1}$ and by the descendant subgraphs of nodes $2, \ldots, n$, over all spanning star forests of $H$ in which node 1 is in state $p$, node $n$ is in state $q$, and exactly $\ell$ centers are used in these descendant parts.

Finally, we reintroduce the edge $e_n = (n,1)$ and incorporate the descendant subgraph of node 1. Let $W_1(p,\ell)$ denote the minimum weight of the descendant subgraph of node 1 when it is in state $p$ and uses exactly $\ell$ centers, namely $W_1(0,\ell) = \Omega(1,\ell)$, $W_1(1,\ell) = \Psi(1,\ell)$, and $W_1(2,\ell) = \Phi(1,\ell)$. By enumerating all feasible ways of including or excluding $e_n$ and combining them with admissible endpoint states and center counts, we obtain the values $\Omega(N,\ell)$, $\Psi(N,\ell)$, and $\Phi(N,\ell)$.

**Lemma 4** For every $\ell \geq 0$,

$$\Omega(N,\ell) = \min_{\ell_1+\ell_2=\ell} \Big\{ C[0,0,\ell_1] + W_1(0,\ell_2),\ C[1,0,\ell_1] + W_1(1,\ell_2),\ C[2,0,\ell_1] + W_1(2,\ell_2) \Big\},$$

$$\begin{aligned}\Psi(N,\ell) = \min_{\ell_1+\ell_2=\ell} \Big\{ & C[0,1,\ell_1] + W_1(0,\ell_2),\ C[1,1,\ell_1] + W_1(1,\ell_2),\ C[2,1,\ell_1] + W_1(2,\ell_2), \\ & C[2,0,\ell_1] + w_{e_n} + W_1(2,\ell_2) \Big\},\end{aligned}$$

$$\begin{aligned}\Phi(N,\ell) = \min_{\ell_1+\ell_2=\ell} \Big\{ & C[0,2,\ell_1] + W_1(0,\ell_2),\ C[1,2,\ell_1] + W_1(1,\ell_2),\ C[2,2,\ell_1] + W_1(2,\ell_2), \\ & C[1,0,\ell_1] + w_{e_n} + W_1(0,\ell_2),\ C[2,0,\ell_1] + w_{e_n} + W_1(0,\ell_2) \Big\}.\end{aligned}$$

*Proof* The proof follows the same reasoning as that of Lemma 2 in Phase I, and considers the DP on a cycle pseudo-node, including the construction of the tables $C$ and $W_1$, as described above, the only difference in Phase II is that the exact number of centers must be enforced.

Consider any solution obtained by combining a solution on the path $H$ with a solution on the descendant subgraph of node 1, and let $\ell_1$ and $\ell_2$ denote the numbers of centers used in these two parts, respectively. By definition of the tables $C[p, q, \ell_1]$ and $W_1(p, \ell_2)$, the total number of centers in the solution on $S_N$ satisfies $\ell = \ell_1 + \ell_2$, since the representative node $u_N$ is already counted in $\ell_1$ according to its state. As in Phase I, the final solution is obtained by deciding whether the edge $e_n = (u_N, 1)$ is selected and by combining the corresponding endpoint states: if $e_n$ is not selected, the states $(s(1), s(u_N))$ must coincide with those encoded in the table $C$, which yields the cases $s(u_N) = 0$ for $\Omega(N, \ell)$, $s(u_N) = 1$ for $\Psi(N, \ell)$, and $s(u_N) = 2$ for $\Phi(N, \ell)$, with $s(1) \in \{0, 1, 2\}$; if $e_n$ is selected, the endpoint states must be compatible with a star edge, which gives the additional combination $(s(1), s(u_N)) = (2, 0)$ for $\Psi(N, \ell)$, corresponding to $u_N$ becoming a leaf attached to a center at node 1, and the combinations $(s(1), s(u_N)) \in \{(1, 0), (2, 0)\}$ for $\Phi(N, \ell)$, corresponding to $u_N$ becoming a center, in which cases the weight $w_{e_n}$ is added. In all cases, a feasible solution on $S_N$ is obtained by combining a solution on the path graph $H$ and a solution on the descendant subgraph of node 1, where the contribution of the path $H$ is given by $C[p, q, \ell_1]$ with $p = s(1)$ and $q = s(u_N)$, and the contribution of the descendant subgraph of node 1 is given by $W_1(p, \ell_2)$ defined for the same state $p$ of node 1; since these two subgraphs are disjoint, the total weight is their sum, and whenever the edge $e_n = (u_N, 1)$ is selected to connect the two endpoints, the edge weight $w_{e_n}$ is added exactly once.

□

Using the recurrence relations in Lemmas 3 and 4, we now obtain a DP algorithm for each subproblem $(\mathcal{P}_k)$ in the following algorithm. Then, the overall complexity of the BSSFP on cactus graphs is stated in the following theorem.

**Theorem 2** *Let $G = (V, E)$ be a weighted cactus graph. The Pareto front of BSSFP on $G$ can be enumerated in $O(|E||V|^2)$ time and $O(|V|^2)$ space.*

*Proof* To analyze the time and space complexity of our method for solving the BSSFP on cactus graphs, it suffices to consider the complexity of Phase II. We first show that the number of constrained problems $(\mathcal{P}_k)$ that must be solved in Phase II is equal to the number $|NS|$ of non-supported efficient solutions. Consider two consecutive supported efficient solutions $(f_1^i, f_2^i)$ and $(f_1^j, f_2^j)$ with $f_2^i < f_2^j$. The procedure starts with $k = f_2^j - 1$ and solves $(\mathcal{P}_k)$. Since there is no supported efficient solution $(f_1, f_2)$ with $f_2 \in (f_2^i, f_2^j)$, each call returns a rightmost non-supported efficient solution $(f_1^m, f_2^m)$ satisfying $f_2^i < f_2^m \leq k$. This solution is recorded and the bound is updated to $k \leftarrow f_2^m - 1$, and the same process is applied recursively to the remaining interval. By induction on the number $|NS_{ij}|$ of non-supported efficient solutions in the interval $(f_2^i, f_2^j)$, the number of calls to $(\mathcal{P}_k)$ on this interval is exactly $|NS_{ij}|$. Summing over all intervals, the total number of calls in Phase II is $|NS| < |V|$.

We now analyze the cost of solving a single constrained problem $(\mathcal{P}_k)$. The DP tables $\Phi(N, \ell)$, $\Psi(N, \ell)$, and $\Omega(N, \ell)$ are computed for every pseudo-node $N$ of the decomposition tree $\mathcal{T}$ and every $\ell \in \{0, \ldots, |V|\}$ by a bottom-up traversal of $\mathcal{T}$, using the recurrences of Lemmas 3 and 4. For a singleton pseudo-node $N$, combining the tables of its children requires distributing the value $\ell$ among the corresponding subgraphs, which can be done in

$O(\deg_{\mathcal{T}}(N)\,|V|^2)$ time, where $\deg_{\mathcal{T}}(N)$ denotes the degree of $N$ in $\mathcal{T}$. For a cycle pseudo-node corresponding to a cycle of length $L$, the computation of the table $C$ involves $L-1$ min-plus products over the fixed state set $\{0,1,2\}$, each of which takes $O(|V|^2)$ time. Hence, the total time for this pseudo-node is $O(L|V|^2)$. Since the blocks of a cactus graph are edge-disjoint, the sum of the lengths of all cycles is $O(|E|)$, and the sum of the degrees of singleton pseudo-nodes in $\mathcal{T}$ is also $O(|E|)$. It follows that the total time required to compute the DP tables for one subproblem $(\mathcal{P}_k)$ is $O(|E||V|^2)$, and the total space required to store them is $O(|V|^2)$. Once the tables have been computed at the root pseudo-node, the optimal value of $(\mathcal{P}_k)$ can be obtained in constant time. Since Phase II evaluates exactly $|NS| < n$ such subproblems and the DP tables are reused across all calls, the overall complexity of enumerating the Pareto front is $O(|E||V|^2)$ time and $O(|V|^2)$ space. □

# 4 Conclusion and discussion

This paper extends the study of the Bi-objective Spanning Star Forest Problem (BSSFP) from trees to cactus graphs. Existing approaches developed for trees cannot be applied to cactus graphs, since the presence of cycles breaks the subtree independence property that these methods rely on. To address this challenge, we develop an exact two-phase dynamic programming (DP) algorithm that enumerates the Pareto front of the BSSFP on cactus graphs in $O(|E||V|^2)$ time. More precisely, we decompose the cactus graph into tree components and cycle components. We handle the tree components by using standard bottom-up DP recurrences. For cycle components, this approach is no longer valid due to the circular dependencies between local decisions. Our key idea is to resolve these dependencies by combining all feasible local states on a cycle through an exact min-plus product, which enforces global consistency for DP and allows the cycle to be handled exactly.

Since the number of centers is at most $|V|$, one possible strategy is to solve a sequence of mono-objective problems obtained by fixing the number of centers $k$ to each value in $\{1,\ldots,|V|\}$, and then to compare the resulting solutions. However, this approach implicitly assumes that the problem with a fixed number of centers can be solved efficiently on cactus graphs, which has not been addressed in the literature so far. In fact, solving the minimum weighted SSF with a fixed number of centers $k$ on cactus graphs can be achieved by adapting the procedure used in Phase II, as developed in this paper. Consequently, this strategy would still rely on our algorithm and would be computationally wasteful. In practice, the number of efficient solutions may be much smaller than $|V|$, and solving a large number of these subproblems would introduce an overhead without providing additional relevant solutions. This bound on the number of efficient solutions motivates the following conjecture: for classes of bi-objective combinatorial optimization problems in which the number of efficient solutions is linear with the input size and the corresponding mono-objective problem is solvable in polynomial time, the Pareto front of the corresponding bi-objective version can also be solved in polynomial time. Hence, a potential direction for future research is to investigate this conjecture on related problems, such as the bi-objective uncapacitated facility location problem on trees.

## Declarations

**Funding** The authors did not receive support from any organization for the submitted work.

**Conflict of interest** The author declares that they have no conflict of interest

## References

[1] A. Agra, D. Cardoso, O. Cerdeira, and E. Rocha. A spanning star forest model for the diversity problem in the automobile industry. *European Journal of Operational Research*, 2005.

[2] A. V. Aho, J. E. Hopcroft, and J. D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.

[3] M. Aïder, L. Aoudia, M. Baiou, A. R. Mahjoub, and V. H. Nguyen. On the star forest polytope for trees and cycles. *RAIRO–Operations Research*, 53, 2019.

[4] S. Athanassopoulos, I. Caragiannis, C. Kaklamanis, and M. Kyropoulou. An improved approximation bound for spanning star forest and color saving. In *Proceedings of the Mathematical Foundations of Computer Science (MFCS)*, volume 5734 of *LNCS*, pages 90–101. Springer, 2009.

[5] O. Briant and D. Naddef. The optimal diversity management problem. *Operations Research*, 52(4):515–526, 2004.

[6] M. Ehrgott. *Multicriteria Optimization*. Springer, 2005.

[7] C. Filippi and G. Stevanato. A two-phase method for bi-objective combinatorial optimization and its application to the tsp with profits. *4OR*, 11(3):253–276, 2013.

[8] G. Mavrotas. Effective implementation of the epsilon-constraint method in multiobjective mathematical programming problems. *Applied Mathematics and Computation*, 213:455–465, 2009.

[9] C. T. Nguyen, J. Shen, M. Hou, L. Sheng, W. Miller, and L. Zhang. Approximating the spanning star forest problem and its applications to genomic sequence alignment. *SIAM Journal on Computing*, 37(3):946–962, 2008.

[10] T. L. Nguyen and V. H. Nguyen. Polynomial-time algorithm for the bi-objective spanning star forest problem on trees. In *Proceedings of the 19th International Conference on Combinatorial Optimization and Applications (COCOA)*, Tianjin, China, Nov. 2025.

[11] V. H. Nguyen. The maximum weight spanning star forest problem on cactus graphs. *Discrete Mathematics, Algorithms and Applications*, 7(3), 2015.

[12] O. Özpeynirci and M. Köksalan. An exact algorithm for finding extreme supported non-dominated points of multiobjective mixed integer problems. *Management Science*, 56(12):2302–2315, 2010.

[13] A. Sedeño-Noda and C. González-Martín. An algorithm for the biobjective integer minimum cost flow problem. *Computers and Operations Research*, 28(2):139–156, 2001.

[14] E. L. Ulungu and J. Teghem. The two phases method: An efficient procedure to solve bi-objective combinatorial optimization problems. *Foundations of Computing and Decision Sciences*, 20:149–165, 1995.